

CORRIDOR HILL RESIDENCE

Smart Maintenance Reporting System

Initial Project Scope • SDLC Foundation • Functional Vision • Security Baseline

STUDENT INNOVATION CHALLENGE

A digital maintenance ecosystem designed around verified residence access, faster issue reporting, intelligent work allocation, transparent resolution tracking, and accountable administration.

Document status: Initial planning baseline

Version: 1.0 • Date: 16 September 2026

Purpose: Establish a clear, buildable scope before UI implementation and detailed technical design.

1. Executive Vision

The Corridor Hill Residence Smart Maintenance Reporting System will replace fragmented, informal maintenance reporting with a single digital workflow connecting verified students, maintenance staff, and residence administrators.

The core innovation is not simply putting a complaint form online. The system should create a complete lifecycle: verify → report → classify → prioritise → assign → work → verify resolution → close → learn.

The platform should be simple enough for a student to report a leaking tap in under a minute, while giving maintenance teams and administrators the operational visibility needed to reduce lost requests, duplicated work, unclear ownership, and unresolved issues.

Design principle: simple for students, operational for maintenance, intelligent for administrators, secure by default.

1.1 Problem Statement

- Maintenance issues can be reported through channels that are difficult to track consistently.
- Requests can lack sufficient information, photos, location details, or urgency context.
- Students may not know whether an issue has been received, assigned, or resolved.
- Maintenance teams need a structured queue rather than scattered messages or paper-based requests.
- Residence management needs measurable evidence of response times, recurring faults, workload, and unresolved issues.

1.2 Project Objective

Build a secure, residence-specific digital maintenance platform that enables authorised Corridor Hill students to report issues and enables maintenance staff and administrators to manage those issues from submission through verified resolution.

1.3 Innovation Goal

For the challenge, differentiation should come from the workflow and intelligence around maintenance, not from adding random features. Every enhancement should reduce time, improve data quality, increase accountability, or improve safety/security.

2. Initial Scope

The first release should focus on one residence and one high-value problem: maintenance issue management. The architecture should be extensible, but the MVP must remain achievable within the challenge timeline.

In Scope	Out of Scope for MVP
Verified Corridor Hill student access	Full residence management / accommodation allocation
First-time password setup and secure login	Payments or residence fee management
Maintenance issue submission with category, location, description and evidence	General social/community platform
Issue status, tracking and student notifications	Complex predictive maintenance requiring large historical datasets

In Scope	Out of Scope for MVP
Maintenance staff queue, assignment and work updates	Integration with every TUT system from day one
Admin dashboard, user management and reporting	Native mobile apps as a mandatory first release
Audit trail, role-based access and security controls	Unbounded AI features without measurable operational value

MVP success condition: a real maintenance request can enter the platform and be traceable to a responsible staff member, a resolution action, and a final closure state.

3. User Roles & Access Model

Role	Primary Responsibilities	Key Permissions
Student	Report and track maintenance issues affecting their room/residence areas.	Create reports, view own reports, add evidence/comments, confirm or dispute resolution.
Maintenance Staff	Receive, inspect, update and resolve assigned maintenance work.	View assigned/eligible jobs, accept work, update status, add notes/evidence, mark work complete.
Admin	Operate and oversee the entire system.	Manage users, staff, categories, priorities, assignments, escalations, reports, audit logs and system settings.

Principle: least privilege. Users should only see and perform actions necessary for their role. Administrative powers must not be granted merely because a user is a staff member.

4. Verified Student Access & Onboarding

The proposed student onboarding model is a strong starting point: students should not freely create residence accounts. Access should be based on an authoritative Corridor Hill resident/student dataset.

4.1 Recommended Registration Flow

Step	System Behaviour
1. Student number	Student enters their TUT student number.
2. Resident validation	System checks whether the student number exists in the authorised Corridor Hill resident dataset and whether the record is active/eligible.
3. Account state	If eligible and not activated, continue to first-time account setup. If already activated, direct the student to normal login/recovery.
4. Secure password setup	Student creates a password meeting the configured security policy. Password is stored only as a strong one-way hash.
5. Authentication	Future logins use student number + password or an approved institutional identity provider if later integrated.
6. Session	System creates a secure authenticated session and applies the Student role.

4.2 Important Security Enhancement

A student number existing in a spreadsheet/database should not by itself be treated as sufficient proof of identity. If the residence has access to an authoritative TUT identity service, institutional email verification, or another trusted identity mechanism, it should be used as an additional

verification factor. If such an integration is unavailable, the system should use controlled resident-data validation plus rate limiting and an administrator-assisted recovery process.

4.3 Resident Data

- Import resident records through a controlled admin process rather than allowing users to edit the source dataset.
- Store only the minimum personal data needed for the application.
- Track record status such as active, inactive, checked-out, or blocked where operationally necessary.
- Protect the resident import process and log administrative changes.
- Do not expose the resident list through public APIs or error messages.

5. Maintenance Staff & Administrator Access

5.1 Maintenance Staff Onboarding

- Each staff member receives a unique staff identifier/number.
- Staff accounts should be created or activated by an authorised administrator rather than through unrestricted public registration.
- Capture operational attributes such as maintenance discipline, team, availability, and active/inactive status where required.
- Require a secure password and optionally institutional email/OTP verification.
- Use role-based access control so a staff account cannot access administrative functions simply by changing client-side values.

5.2 Administrator Controls

- Activate, suspend and manage student and staff accounts.
- Manage maintenance categories, priorities, locations and service-level rules.
- Assign or reassign work orders.
- Monitor overdue requests and escalations.
- View audit logs and security events.
- Generate operational reports and export approved datasets.
- Configure notifications and system behaviour without modifying application code.

6. Core Maintenance Workflow

Every report should have a clear state machine. This prevents requests from disappearing into a generic 'submitted' bucket.

Status	Meaning	Typical Actor
Submitted	Issue has been captured and stored.	System / Student
Triaged	Issue has been classified and priority assessed.	Admin / Dispatcher
Assigned	A responsible maintenance staff member/team is identified.	Admin / Dispatcher
Accepted	Assigned staff member has acknowledged the job.	Maintenance
In Progress	Physical work/inspection is underway.	Maintenance

Status	Meaning	Typical Actor
Awaiting Parts / Access	Work cannot continue because of a documented dependency.	Maintenance
Resolved	Maintenance work is reported as completed, with evidence where applicable.	Maintenance
Verified / Closed	Student or authorised staff confirms the issue is resolved, or the system closes it according to policy.	Student / Admin
Reopened	Issue persists or was incorrectly marked resolved.	Student / Admin

6.1 Student Report Form

- Maintenance category: plumbing, electrical, furniture, appliance, network, building, cleaning-related fault, safety, other.
- Specific location: building/area/floor/room or common-area location.
- Short issue title and plain-language description.
- Photo/video evidence where useful.
- Optional voice-to-text or guided prompts as a future accessibility enhancement.
- Availability/access information where a technician needs entry.
- Automatic timestamp and unique issue/reference number.

7. Differentiating Features

These features are designed to make the project feel like a complete operational product rather than a CRUD complaint form.

Feature	How It Works	Value
Smart Triage	Use category, location, severity indicators and issue history to suggest priority and routing. Admin retains control.	Reduces manual sorting and improves consistency.
Duplicate Detection	Flag potentially similar open reports for the same area/category before a new report becomes duplicate work.	Prevents repeated requests for the same fault.
Live Status Timeline	Students see a clean timeline of what happened to their request without exposing internal/private notes.	Improves transparency and reduces 'is anyone handling this?' questions.
Maintenance Work Queue	Staff see available/assigned work with priority, location, age, evidence and action buttons.	Makes the platform useful to technicians, not only students.
Escalation Engine	Automatically flags jobs approaching or exceeding configured service targets.	Makes overdue work visible before it becomes forgotten.
Resolution Evidence	Technician can attach completion notes/photos; student can confirm or reopen.	Creates accountability and a stronger audit trail.
Recurring Fault Intelligence	Aggregate historical issues by location/category to reveal repeated failures.	Turns maintenance data into residence insight.
Maintenance Heatmap	Visualise issue concentration by residence location, without exposing sensitive student information.	Helps management identify problematic areas.

Feature	How It Works	Value
Announcement/Incident Mode	If one known outage affects many rooms, admins can link related reports to a master incident.	Avoids treating one building-wide fault as 30 unrelated jobs.

8. Security & Privacy Baseline

Security should be presented as a core product requirement, not an optional enhancement.

Control	Implementation Direction
Authentication	Secure password hashing; protected sessions/tokens; account activation states; secure password recovery.
Authorisation	Server-side RBAC for Student, Maintenance and Admin; never rely on hidden UI controls alone.
Input validation	Validate and constrain all request fields, file metadata, identifiers and state transitions on the server.
Rate limiting	Limit repeated login, first-time activation, password reset and suspicious request attempts.
Account enumeration protection	Avoid revealing whether a student number is valid through overly specific public error messages.
File security	Restrict file types/sizes, scan or validate uploads, store outside executable paths, and use controlled access to evidence.
Audit logging	Record sensitive actions such as account changes, assignments, status changes, admin actions and security events.
Database security	Use parameterised queries/ORM protections, least-privilege database credentials, backups and restricted network access.
Transport security	HTTPS/TLS for production traffic; secure cookies; avoid transmitting credentials through insecure channels.
Secrets	Keep credentials/API keys in environment or secret-management facilities, never source code or public repositories.
Privacy	Collect minimum necessary personal information, restrict access, define retention, and align handling with applicable TUT/residence rules and South African privacy requirements.
Operational recovery	Backups, restore testing, error handling and incident-response procedures.

Security target: design against common web risks such as broken access control, injection, authentication failures, insecure file handling, excessive data exposure, and security misconfiguration. The final implementation should be tested rather than merely claiming compliance.

9. Notification & Communication Layer

- Student receives confirmation when a report is submitted.
- Student receives notifications when status changes or additional information is requested.
- Maintenance staff receive new/assigned job notifications.
- Admins receive escalation alerts and system/security alerts.
- Use in-app notifications as the primary system of record, with email/SMS/push as optional delivery channels.
- Notification preferences should be configurable where appropriate.

- Avoid putting sensitive information into notification previews.

10. SDLC Roadmap

The project should be built through a disciplined Software Development Life Cycle rather than jumping directly into screens.

Phase	Outputs for Corridor Hill
1. Planning	Problem statement, objectives, stakeholders, assumptions, constraints, project scope, initial risk register.
2. Requirements Analysis	Functional requirements, non-functional requirements, user stories, acceptance criteria, role/permission matrix.
3. System Design	Architecture, database ERD, API design, authentication flow, state machine, security design, wireframes/UI system.
4. Development	Frontend, backend/API, database, authentication, file handling, notifications, dashboards and integrations.
5. Testing	Unit tests, integration tests, role/permission tests, authentication tests, file-upload tests, usability tests, security testing.
6. Deployment	Production environment, database migrations, HTTPS, secrets, backups, monitoring and release checklist.
7. Maintenance	Bug fixes, monitoring, user feedback, performance improvements, security updates and measured feature iterations.

Recommended build order: requirements → data model → authentication/authorisation → maintenance workflow → dashboards → notifications → intelligence → hardening/testing → deployment.

11. Initial Functional Requirements

ID	Requirement	Priority
FR-01	System shall validate a student's eligibility against the authorised Corridor Hill resident dataset.	Core
FR-02	System shall support first-time password setup for eligible students.	Core
FR-03	System shall authenticate returning users securely.	Core
FR-04	System shall enforce role-based access for students, maintenance staff and administrators.	Core
FR-05	Students shall create maintenance reports with structured information and optional evidence.	Core
FR-06	System shall generate a unique reference for every maintenance request.	Core
FR-07	Maintenance staff shall view and manage assigned work.	Core
FR-08	Administrators shall manage users, requests, assignments and system configuration.	Core
FR-09	System shall maintain an auditable history of important actions and status changes.	Core
FR-10	System shall notify relevant users of important workflow events.	Core
FR-11	System should detect or flag likely duplicate/related requests.	Enhancement
FR-12	System should provide operational analytics and recurring-fault insights.	Enhancement

11.1 Non-Functional Requirements

- Security: strong authentication, RBAC, validation, auditability and secure file handling.
- Usability: student issue submission should be short, mobile-friendly and understandable without technical language.
- Performance: common dashboard and submission actions should respond promptly under expected residence load.
- Availability: production service should have monitoring, backups and documented recovery procedures.
- Maintainability: modular codebase, migrations, environment configuration, logging and API documentation.
- Accessibility: readable typography, keyboard accessibility, contrast, labels and responsive layouts.
- Scalability: architecture should allow additional residences later without rewriting the core workflow.

12. Initial Data Model

A relational database is recommended because the platform contains structured relationships, workflow states, permissions, audit history and reporting requirements.

Entity	Purpose
Student	Authorised resident identity and account state.
Staff	Maintenance worker profile, staff number, discipline and status.
Admin / User	Administrative identity and role mapping.
Residence / Location	Buildings, blocks, floors, rooms and common areas.
Maintenance Request	Main issue record, reporter, location, category, priority and current status.
Request Status History	Every significant state transition with actor and timestamp.
Assignment	Links work to staff/team and records assignment history.
Attachment / Evidence	Photos or other permitted evidence linked to requests.
Comment / Note	Student-facing comments and controlled internal work notes.
Notification	Tracks system-generated notification events and delivery state.
Audit Log	Security and accountability record for sensitive actions.
Incident	Optional master record linking multiple requests to one building-wide fault.
Maintenance Category / Rule	Configurable classification, priority and service targets.

13. Proposed High-Level Architecture

A web-first responsive application is recommended for the challenge because it can serve students, technicians and administrators from one deployable system while remaining accessible on phones and computers.

Layer	Responsibility
Client / Web App	Responsive student, maintenance and admin interfaces; client-side UX only, never authoritative security.
Application/API	Authentication, authorisation, maintenance workflow, validation, assignment, notifications and reporting APIs.

Layer	Responsibility
Database	Users, residents, requests, statuses, assignments, evidence metadata, notifications and audit data.
Object/File Storage	Protected storage for maintenance photos/evidence with controlled access.
Notification Services	Email/SMS/push delivery where selected.
Monitoring & Logging	Application errors, security events, performance and operational health.

Future-ready direction: keep residence-specific data and configuration separated logically so the same platform can later support additional residences without creating separate codebases.

14. Analytics & Success Metrics

The platform should prove its impact with measurable operational metrics rather than only screenshots.

- Number of maintenance requests submitted.
- Percentage of requests successfully assigned.
- Average time from submission to triage.
- Average time from assignment to first staff action.
- Average time to resolution.
- Number/percentage of overdue requests.
- Reopened-request rate.
- Recurring faults by location/category.
- Requests grouped under major incidents.
- Student confirmation/feedback after resolution.

These metrics can later form an admin dashboard and a challenge demonstration story: the system does not merely record complaints; it creates measurable operational visibility.

15. Initial Risk Register

Risk	Impact	Mitigation
Resident spreadsheet is outdated	Unauthorised or blocked students may be incorrectly handled.	Define source-of-truth ownership, import validation, timestamps and admin reconciliation.
Student number alone is treated as identity proof	Account takeover/unauthorised activation risk.	Add trusted identity/email verification where available; rate limit activation.
Maintenance staff do not update statuses	Dashboard becomes inaccurate.	Make workflow fast, mobile-friendly and use reminders/escalations.
Duplicate reports	Wasted maintenance effort.	Duplicate/related-issue detection and master incident workflow.
Poor evidence quality	Technicians cannot diagnose issues remotely.	Guided reporting fields and optional photo evidence.
Overbuilding for the challenge	Core system remains unfinished.	Freeze MVP scope and add enhancements only after core workflow is stable.

Risk	Impact	Mitigation
Sensitive data exposure	Privacy and security incident.	Least privilege, secure storage, access controls, logging, testing and data minimisation.

16. Proposed MVP Delivery Sequence

Milestone	Deliverable
M1 — Foundation	Requirements, user stories, architecture, database schema, repository structure and development environments.
M2 — Identity	Student resident validation, first-time activation, login, staff onboarding and RBAC.
M3 — Student Reporting	Issue form, evidence upload, reference number, report history and status timeline.
M4 — Maintenance Operations	Staff queue, assignment, accept/start/complete workflow, notes and evidence.
M5 — Administration	Admin dashboard, user management, assignment controls, categories, priorities and audit logs.
M6 — Intelligence	Duplicate detection, incident linking, escalation rules and operational analytics.
M7 — Hardening	Security testing, role testing, error handling, backups, monitoring, accessibility and performance checks.
M8 — Demonstration	End-to-end challenge scenario, metrics, architecture explanation, security story and polished UX.

17. Immediate Next Steps

Before writing production code, the team should produce the following artefacts in order:

- Stakeholder and user-role map.
- Detailed problem statement and project objectives.
- Functional and non-functional requirements specification.
- User stories and acceptance criteria for Student, Maintenance Staff and Admin.
- Use-case diagram.
- System architecture diagram.
- Entity Relationship Diagram (ERD).
- Authentication and authorisation flow diagrams.
- Maintenance request state/workflow diagram.
- API endpoint specification.
- Security threat model and security test checklist.
- Low-fidelity wireframes followed by the final UI design system.
- Implementation backlog divided into MVP, enhancement and future phases.

The next planning stage should turn this initial scope into a complete SDLC specification with traceable requirements: every feature should map to a user need, database/API behaviour, UI flow, acceptance

criterion and test case.